

UNIVERSITE CHEIKH ANTA DIOP DE DAKAR



ECOLE SUPERIEURE POLYTECHNIQUE DE DAKAR



DEPARTEMENT : GENIE INFORMATIQUE

FILIERE : LICENCE 2 TELECOMS & RESEAUX

ANNEE ACADEMIQUE : 2025-2026

PROJET DE FIN DE MODULE APPLICATION CLIENT & SERVEUR

PROJET DE MISE EN PLACE DE LA PLATEFORME SMARTTECHCONNECT

Réalisé Par :

SERIGNE ABDOURAHMANE SAMB

Encadrant :

DR K.GUEYE

PLAN :

INTRODUCTION

I. PRESENTATION DE L'ENVIRONNEMENT & CONFIGURATION DE LA BASE DE DONNEES

II. REALISATIONS

- a) **Authentification**
- b) **Chat en temps réel**
- c) **Tableau blanc collaboratif**
- d) **Appels vidéo/audio P2P**

III. DIFFICULTES RENCONTREES

CONCLUSION

INTRODUCTION

Ce travail s'est déroulé dans le cadre du module Application Client/Serveur, il s'agit en fait de la conception du projet Smarttechconnect, qui est une application web collaborative pour étudiants.

L'idée est de mettre en œuvre tous les contenus du cours pratique tels que l'architecture client-serveur, les sessions sécurisées, la communication en temps réel avec Socket.IO et la vidéoconférence peer-to-peer en utilisant WebRTC/PeerJS.

Il s'agit d'une application full-stack.NET qui se compose d'un serveur Node.js utilisant Express.js et une base de données MySQL, ainsi que des spécificités de travail temp réel entre utilisateurs.

I. MISE EN PLACE DE L'ENVIRONNEMENT & CONFIGURATION

a) Initialisation du projet Node.js et installation des dépendances

Tout d'abord nous allons entrer la commande `npm init -y`, qui permet de créer le fichier `package.json` contenant les informations du projet ainsi que les dépendances `express`, `socket.io`, `mysql2`, `bcrypt`, `express-session`, `peerjs` nécessaires à son développement.

```
C:\Users\Abdourahmane\Documents\smarttechconnect>npm init -y
Wrote to C:\Users\Abdourahmane\Documents\smarttechconnect\package.json:

{
  "name": "smarttechconnect",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
C:\Users\Abdourahmane\Documents\smarttechconnect>npm install express socket.io mysql2 bcrypt express-session peerjs ejs
added 113 packages, and audited 114 packages in 13s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Abdourahmane\Documents\smarttechconnect>npm install --save-dev nodemon
added 26 packages, and audited 140 packages in 7s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Abdourahmane\Documents\smarttechconnect>
```

b) Configuration de la base de données MySQL

Par la suite, nous avons utilisé MySQL via l'application XAMP disponible sur Windows pour stocker toutes les données de l'application tels que les comptes utilisateurs, les messages échangés et les salons de discussion.

- ✓ Création de la base de données :

```
MariaDB [(none)]>
MariaDB [(none)]> create database smarttechconnect;
Query OK, 1 row affected (0.002 sec)
```

- ✓ Création de la table identifiant utilisateur(nom, adresse mail, mot de passe) :

```
MariaDB [(none)]> use smarttechconnect;
Database changed
MariaDB [smarttechconnect]>
MariaDB [smarttechconnect]> create table if not exists users (
->     id int primary key auto_increment,
->     username varchar(50) unique not null,
->     email varchar(100) unique not null,
->     password varchar(255) not null,
->     avatar_color varchar(7) default '#3b82f6',
->     is_online boolean default false,
->     last_seen timestamp null,
->     created_at timestamp default current_timestamp,
->     index idx_username (username),
->     index idx_email (email),
->     index idx_online (is_online)
-> ) engine=innodb default charset=utf8mb4;
Query OK, 0 rows affected (0.354 sec)
```

- ✓ Création des salons :

```
MariaDB [smarttechconnect]>
MariaDB [smarttechconnect]> create table if not exists rooms (
->     id int primary key auto_increment,
->     name varchar(100) unique not null,
->     description text,
->     icon varchar(50) default '🗨️',
->     created_by int,
->     created_at timestamp default current_timestamp,
->     foreign key (created_by) references users(id) on delete set null,
->     index idx_name (name)
-> ) engine=innodb default charset=utf8mb4;
Query OK, 0 rows affected (0.229 sec)
```

- ✓ Création des sections de messageries :

```
MariaDB [smarttechconnect]>
MariaDB [smarttechconnect]> create table if not exists messages (
  ->   id int primary key auto_increment,
  ->   user_id int,
  ->   room_id int not null,
  ->   content text not null,
  ->   message_type enum('text', 'image', 'file') default 'text',
  ->   created_at timestamp default current_timestamp,
  ->   foreign key (user_id) references users(id) on delete set null,
  ->   foreign key (room_id) references rooms(id) on delete cascade,
  ->   index idx_room (room_id),
  ->   index idx_user (user_id),
  ->   index idx_created (created_at)
  -> ) engine=innodb default charset=utf8mb4;
Query OK, 0 rows affected (0.241 sec)
```

- ✓ Création de la table pour la gestion des appels :

```
MariaDB [smarttechconnect]>
MariaDB [smarttechconnect]> create table if not exists calls (
  ->   id int primary key auto_increment,
  ->   caller_id int,
  ->   receiver_id int,
  ->   start_time timestamp default current_timestamp,
  ->   end_time timestamp null,
  ->   status enum('initiated', 'ongoing', 'ended', 'missed') default 'initiated',
  ->   foreign key (caller_id) references users(id) on delete set null,
  ->   foreign key (receiver_id) references users(id) on delete set null,
  ->   index idx_caller (caller_id),
  ->   index idx_receiver (receiver_id)
  -> ) engine=innodb default charset=utf8mb4;
Query OK, 0 rows affected (1.007 sec)
```

- ✓ Insertion de valeur pour les salons :

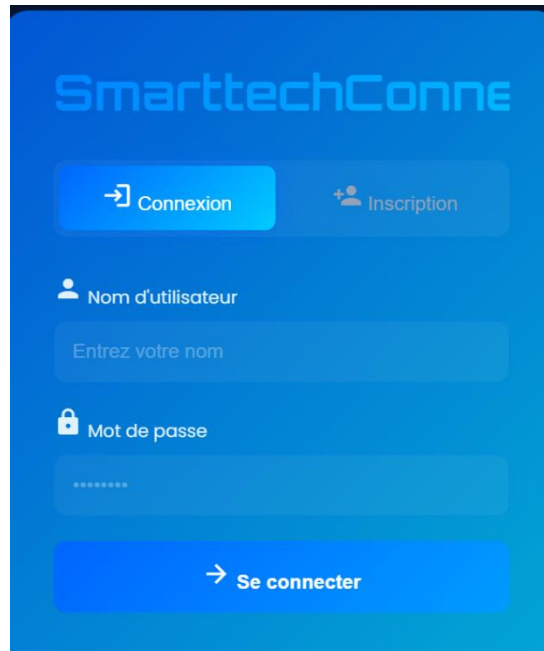
```
MariaDB [smarttechconnect]>
MariaDB [smarttechconnect]> insert into rooms (name, description, icon) values
  -> ('general', 'salon principal de discussion', '🌐'),
  -> ('entraide', 'entraide sur les projets', '💡'),
  -> ('techtalk', 'discussions techniques', '💻'),
  -> ('detente', 'espace de relaxation', '🎮'),
  -> ('projets', 'collaboration sur les projets', '🚀');
Query OK, 5 rows affected (0.136 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

II. REALISATIONS

Cette section détaille la mise en œuvre concrète des différentes fonctionnalités de Smarttechconnect, en expliquant les choix techniques et le fonctionnement de chaque module.

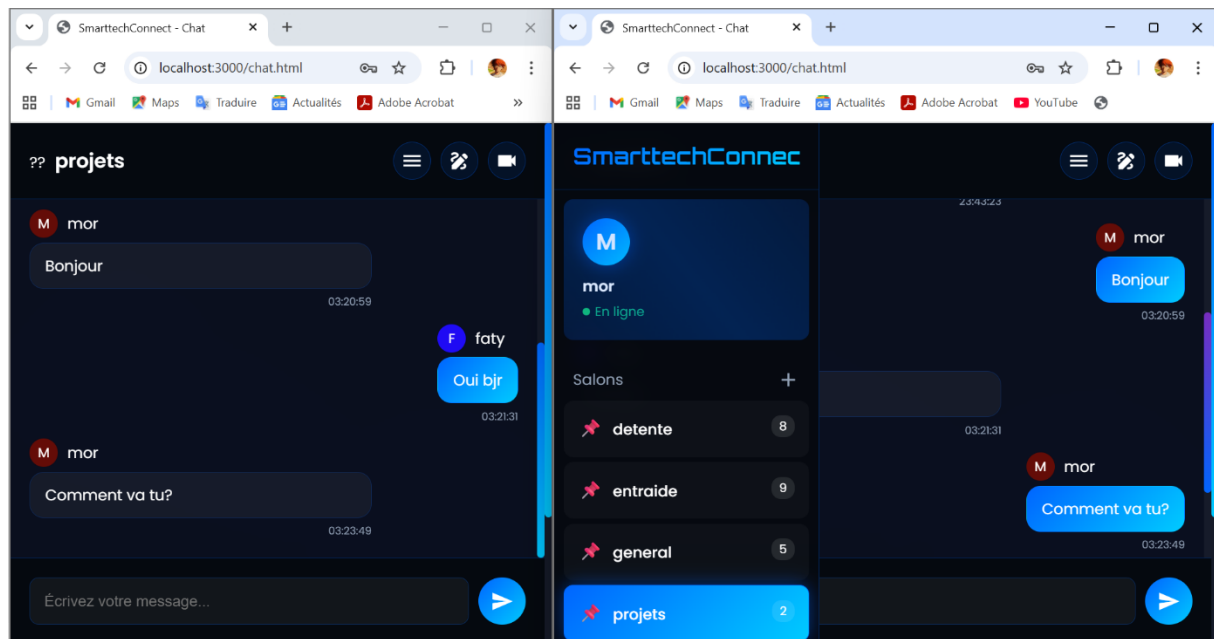
a) Authentification

L'authentification est gérée avec express-session afin de sécuriser les sessions des utilisateurs. Les mots de passe sont chiffrés avec bcrypt, et l'accès aux pages comme le chat et la vidéo est réservé aux utilisateurs connectés.



b) Chat en temps réel :

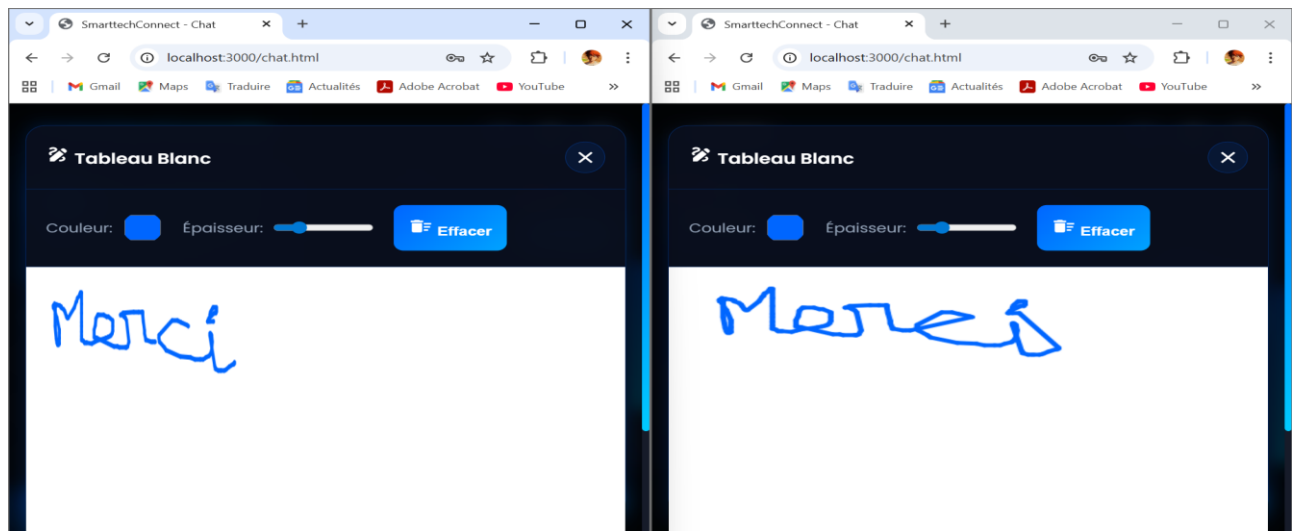
Le chat est l'une des fonctionnalités principales de l'application. Il utilise Socket.IO pour permettre l'échange de messages en temps réel et MySQL pour enregistrer les messages. Lorsqu'un utilisateur rejoint un salon, il est ajouté à une room et les messages envoyés sont partagés avec tous les participants, avec une notification en cas de déconnexion d'un utilisateur.



c) Tableau Blanc Collaboratif

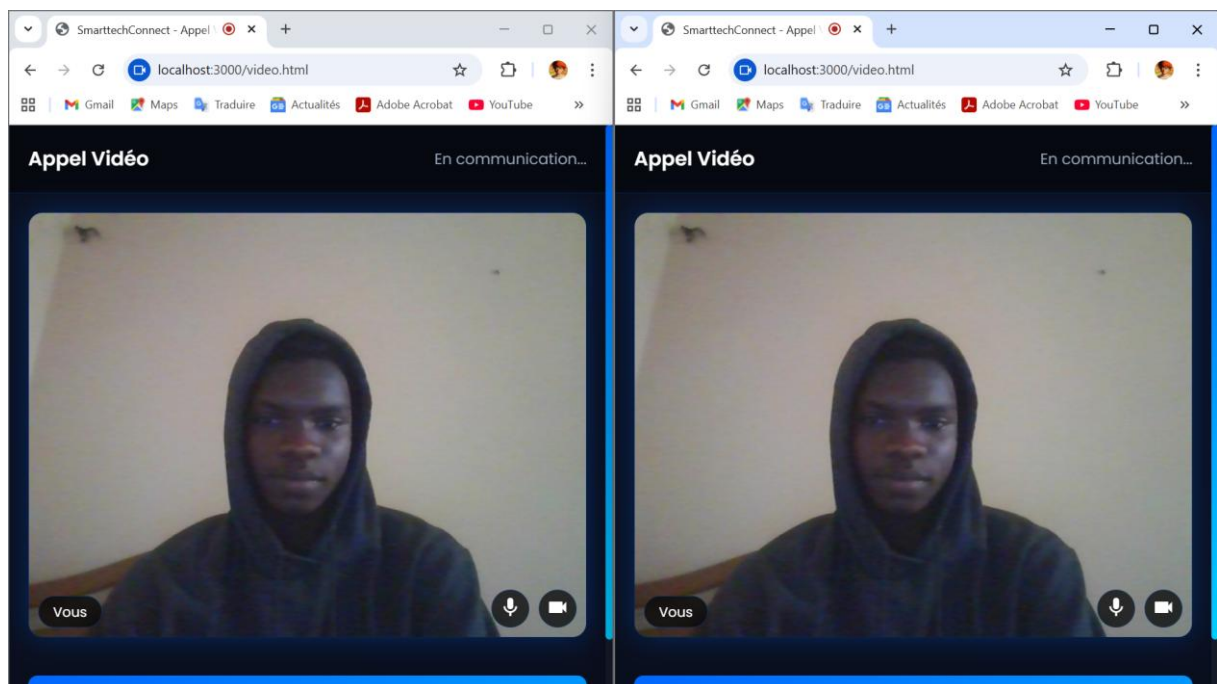
Le tableau blanc collaboratif permet à plusieurs utilisateurs d'un même salon de dessiner simultanément. Il a été implémenté avec l'élément HTML5 <canvas> et synchronisé en temps réel via Socket.IO. L'utilisateur peut choisir une couleur et une épaisseur de trait, et chaque mouvement de souris est capturé et émis vers le serveur via l'événement draw, ensuite le serveur redistribue l'événement à tous les autres participants du salon, qui reproduisent le tracé sur leur propre canvas.

Le bouton "Effacer" permet de vider le tableau, action également synchronisée.



d) Appels vidéo/audio P2P

Les appels vidéo et audio sont réalisés en pair-à-pair grâce à PeerJS. Chaque utilisateur possède un identifiant unique qui permet d'établir une communication directe entre deux personnes. Le flux vidéo et audio est capturé avec `getUserMedia`, puis affiché pour les deux utilisateurs, avec des options pour couper le micro, désactiver la caméra ou terminer l'appel.



III . DIFFICULTES RENCONTREES

Tout au long du développement de SmarttechConnect, plusieurs obstacles techniques ont dû être surmontés. Voici les principaux problèmes rencontrés et les solutions mises en œuvre.

1) Problèmes d'installation et de configuration :

Au démarrage du projet, l'absence du module peer a provoqué une erreur bloquante. Nous avons bien installé peerjs pour le client, mais le serveur nécessitait également le module peer. La solution a été simple : exécuter `npm install peer` pour ajouter la dépendance manquante. Par ailleurs, les fichiers CSS ne se chargeaient pas au premier lancement car les chemins dans les pages HTML étaient incorrects. Nous avons corrigé les liens en plaçant tous les fichiers statiques dans le dossier public et en utilisant des chemins relatifs appropriés.

2) Optimisation des performances du chat

Au début, les messages mettaient près d'une seconde à apparaître après envoi. En analysant le code, nous avons constaté que le serveur effectuait trois requêtes SQL distinctes. Cette redondance ralentissait considérablement le traitement.

Nous avons donc optimisé le processus en utilisant une seule requête avec sous-requête pour l'insertion, et en émettant immédiatement le message avec les données déjà disponibles côté client. Cette modification a réduit la latence à quelques millisecondes seulement.

3) Connexion Socket.io lente

Par moment, la connexion Socket.IO utilisait le mécanisme des requêtes HTTP répétées au lieu du WebSocket, ce qui entraîne une latence plus élevée et une surcharge réseau. Nous

avons résolu ce problème en forçant l'utilisation du transport WebSocket dans la configuration du client :

```
const socket = io({ transports: ['websocket'] });
```

Ainsi, la communication est devenue instantanée et plus fiable.

CONCLUSION

Le projet SmarttechConnect a permis de développer une plateforme collaborative complète répondant à toutes les exigences du cahier des charges.

L'application offre une authentification sécurisée, un chat en temps réel avec persistance des messages, un tableau blanc collaboratif synchronisé et des appels vidéo en pair-à-pair.

Les objectifs pédagogiques du module ont été atteints : mise en pratique des modèles réseau, maîtrise des sockets avec Socket.IO, développement d'interfaces dynamiques, intégration de WebRTC pour la vidéo et gestion d'une base de données MySQL.

Ce travail a également été l'occasion de surmonter plusieurs défis techniques, comme l'optimisation des requêtes ou la configuration du serveur Peer.

L'application fonctionne de manière fluide en local et remplit son rôle d'outil collaboratif pour les étudiants. Des améliorations futures pourraient inclure le chiffrement des communications, une version mobile ou l'ajout de fonctionnalités comme le partage d'écran.

Ce projet nous a permis de consolider nos compétences en développement full-stack et de mieux comprendre les enjeux des applications temps réel et des communications P2P.

FIN

